

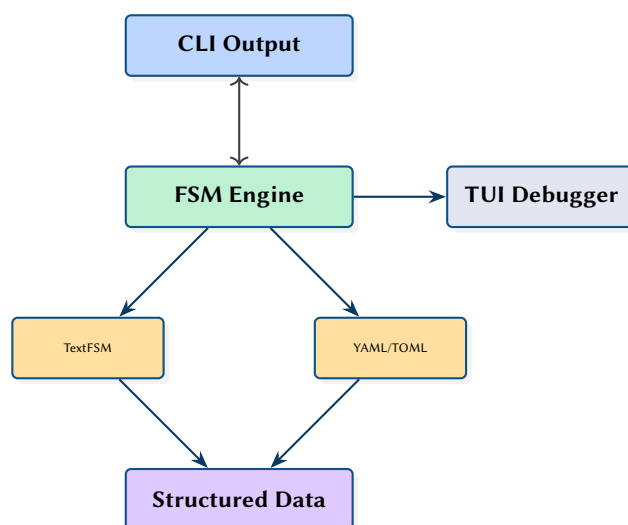
cliscrape: A Rust CLI Parsing Engine with Interactive TUI Debugger

TextFSM-Compatible Parsing, Modern Templates, and Network-Compiler Foundations

Simon Knight

April 2026 • Version 0.1.2 report revision

Last updated: April 12, 2026



Abstract. Network automation pipelines rely heavily on parsing unstructured CLI output from network devices. This report presents cliscrape, a Rust implementation of a TextFSM-compatible state machine parser. Historical benchmarks have reported multi-million-line-per-second throughput, but release performance claims should be regenerated from the current benchmark suite before publication. The current implementation supports legacy TextFSM templates, a modern YAML/TOML template format, embedded/XDG template resolution, structured logging, policy constraints, semantic diffing, grammar induction, best-effort synthetic generation, and an interactive TUI debugger. Full NTC-Templates coverage, universal ledger schemas, strict oracle semantics, release automation, and streaming parsing remain tracked follow-up work.

Contents

1	Introduction and Ecosystem Context	2
1.1	Ecosystem Role	2
2	Operational Model and Case Studies	2
3	FSM Engine Architecture	3
3.1	Stateful Modifiers	3
4	Template Formats	3
4.1	Native Type Coercion	4
5	Production Hardening	4
5.1	Native Binary Deployment	4
5.2	Embedded Template Library & Hot-Reloading	4
5.3	Validation and Edge Case Management	4
5.4	Structured Observability	5
6	Isomorphic Simulation and Synthetic Generation	5
6.1	The Inverse Parser (FSM^{-1})	5
6.2	Mathematical Verification	5
7	Interactive TUI Debugger	5
8	Future Trajectories: The Network Compiler	6
8.1	Semantic Drift Analysis	6
8.2	Vendor-Neutral State Schemas	6
8.3	Grammar Induction and Structural Inference	6
8.4	The FSM-Oracle: Self-Verification	6
9	Performance Evaluation	6
10	Design Summary	6
	Revision History	8
FSM	Finite State Machine	2
TUI	Terminal User Interface	5
JSON	JavaScript Object Notation	2

1 Introduction and Ecosystem Context

The networking industry relies on CLI-based management for device configuration and monitoring. Parsing this unstructured text output is often the bottleneck in automation pipelines. TextFSM [1] provides a template-based approach using state machines, and the NTC-Templates [2] library offers hundreds of community-maintained templates. `clisrape` implements a compatible TextFSM loading path and a modern template format; full coverage of the external NTC template corpus is still a compatibility target rather than the current embedded library state.

However, the Python TextFSM implementation has three core limitations: performance limitations due to interpreted execution (often struggling with large `show tech-support` outputs), readability issues caused by dense inline regex syntax, and a lack of observability when templates fail to match expected output.

1.1 Ecosystem Role

`clisrape` operates within the Operations Path of the broader network automation ecosystem. It sits directly behind the connectivity layer, transforming raw device output into structured operational state.



Figure 1. The Operations Path pipeline. `deviceinteraction` handles connectivity, while `clisrape` handles pure parsing, feeding operational state to analysis tools.

2 Operational Model and Case Studies

The foundational goal of `clisrape` is taking semi-structured CLI text and enforcing a typed schema contract. Consider a standard `show version` command from a Cisco IOS device:

Raw CLI Output (Cisco IOS)

```
1 Cisco IOS Software, C2960 Software (C2960-LANBASEK9-M), Version 15.0(2)SE11, RELEASE SOFTWARE (fc3)
2 ROM: Bootstrap program is C2960 boot loader
3 SW-CORE-01 uptime is 5 weeks, 2 days, 3 hours, 45 minutes
4 System image file is "flash:/c2960-lanbasek9-mz.150-2.SE11.bin"
```

Instead of brittle custom scripts or string-splitting, the Finite State Machine (FSM) engine applies an embedded YAML or TextFSM template. The variables defined in the template dictate the keys of the resulting JSON object.

Parsed Structured Output (excerpt)

```
1 [
2   {
3     "hostname": "SW-CORE-01",
4     "uptime": "5 weeks, 2 days, 3 hours, 45 minutes",
5     "version": "15.0(2)SE11,"
6   }
7 ]
```

This operational model allows downstream tools to perform type-aware assertions against properties like uptime and version, decoupled from many vendor-specific formatting quirks of the original CLI. The current validation snapshots have also exposed template-specific extraction bugs, such as Cisco IOS serial-number handling, which should be fixed as template correctness work rather than hidden in report examples.

3 FSM Engine Architecture

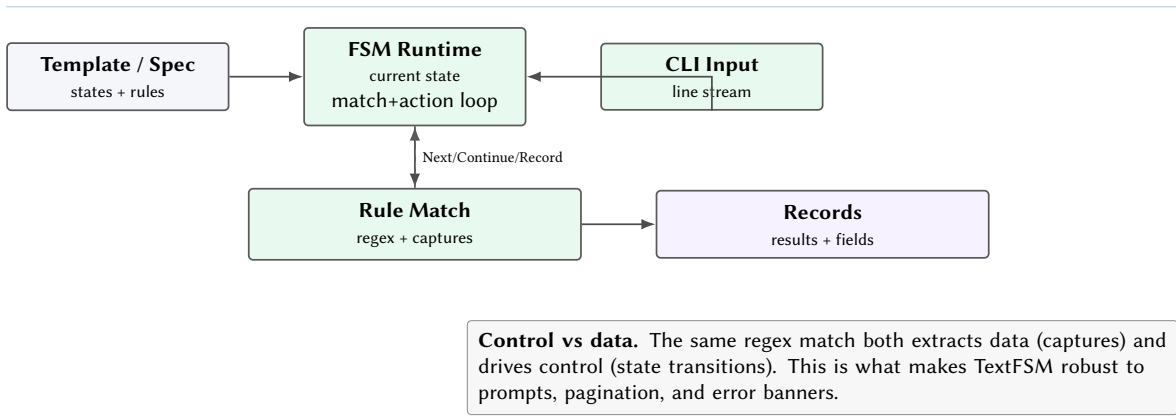


Figure 2. FSM engine model: rule matching is both evidence extraction and control-flow.

The core engine implements the exact TextFSM state machine semantics using Rust.

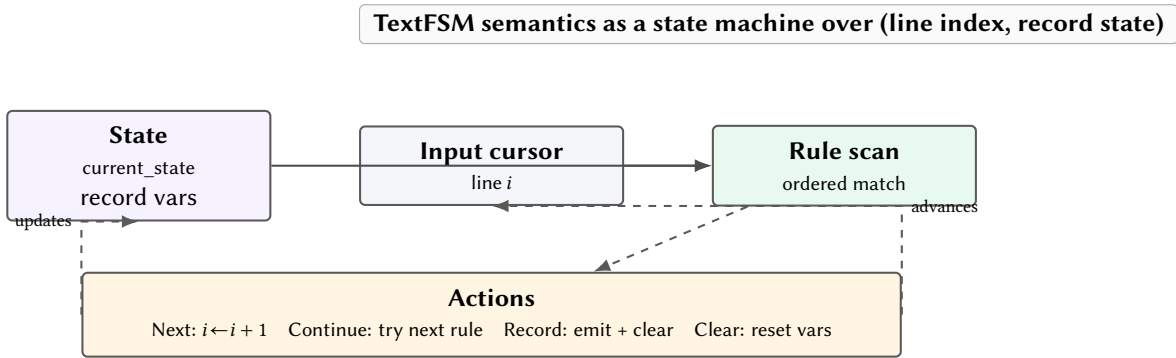


Figure 3. TextFSM parsing as an interpreter over a line cursor and an accumulating record.

For each input line, the engine evaluates rules in the current state sequentially. On match, captured groups are assigned to named variables, and the defined action (Next, Continue, Record, Clear) is executed.

3.1 Stateful Modifiers

Simple regular expressions are insufficient for network CLI outputs that exhibit hierarchical or stateful formatting (e.g., a routing table where a single routing protocol applies to multiple subsequent subnet lines). To support this, the engine implements TextFSM’s stateful variable modifiers:

- **Filldown:** Retains a captured variable’s value across multiple record emissions until explicitly cleared or overwritten.
- **List:** Rather than overwriting a variable on subsequent matches, accumulates the matched values into a native [JSON](#) array.

These modifiers allow the parser to build deep, relational data structures from flat text without requiring complex post-processing.

4 Template Formats

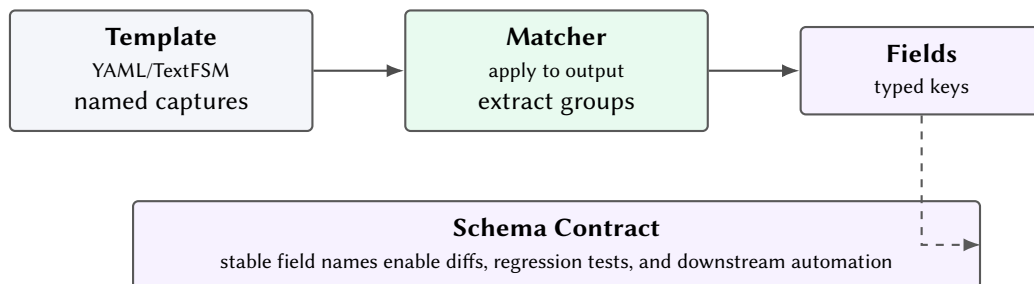


Figure 4. Templates define the parsing contract: semistructured CLI output becomes stable, typed records.

***Insight:**
The Rust regex crate’s DFA-based engine provides predictable $O(n)$ matching per line, avoiding the catastrophic backtracking possible with Python’s `re` module on pathological patterns.*

Supporting both TextFSM and YAML formats allows teams to migrate incrementally. Existing templates work immediately via a compatibility translation layer. New templates can use the more readable YAML format. Both compile to the same internal [FSM](#) representation (IR).

Beyond mere readability, the shift to YAML/TOML introduces explicit schema definition capability. Because values can be natively typed (e.g., explicitly defining an interface MTU as an integer rather than an arbitrary regex string), `cliscape` can support downstream tooling such as JSON Schema generation, typed client bindings, and documentation validation. Those assets are roadmap items, not all current shipped behavior.

4.1 Native Type Coercion

Legacy Python TextFSM implementations fundamentally treat all captured data as strings. If a device outputs an MTU of 1500, the resulting JSON output will be `"mtu": "1500"`. This pushes the burden of data normalisation onto downstream analysis tools.

Because `cliscape`'s modern templates allow explicit type definitions, the engine performs native type coercion during the match phase. An extracted string representing a number is safely converted and emitted as a native [JSON](#) Integer. This seemingly small feature represents a major architectural win for the Operations Path: when data reaches `netassure`, it is instantly queryable without secondary casting operations.

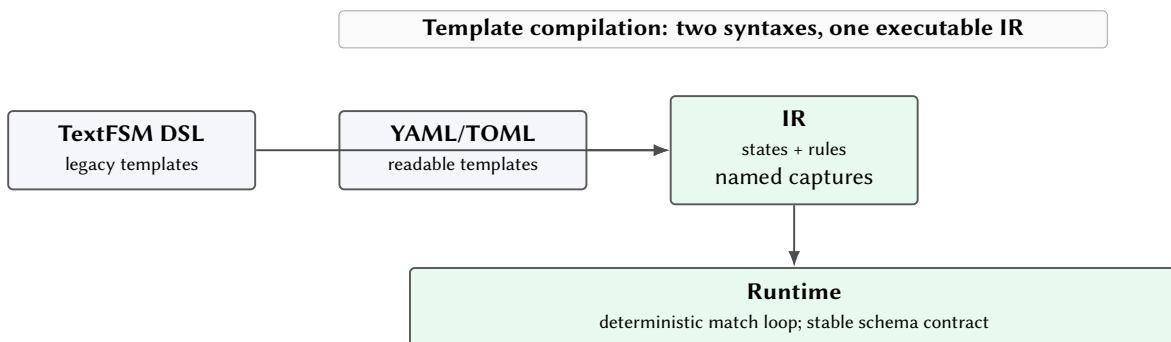


Figure 5. Compilation boundary: both template syntaxes compile into one IR consumed by the same runtime.

5 Production Hardening

Moving from a parsing prototype to a robust operational tool required several critical system enhancements to handle edge cases, simplify deployments, and ensure continuous availability.

5.1 Native Binary Deployment

In enterprise networking environments, deploying tools to jump-hosts or restricted bastion servers is often hindered by Python runtime dependencies, virtual environments, and missing pip packages. Because `cliscape` is written in Rust and bundles its templates internally, it can be distributed as a single native executable per target platform. Operators can copy the executable to a restricted node and parse CLI outputs without installing a Python runtime or template package set.

5.2 Embedded Template Library & Hot-Reloading

To eliminate the friction of distributing raw template files, `cliscape` ships with a small embedded template library. Using an XDG-compliant resolver, users can parse outputs by declaring a template identifier, e.g., `-t cisco_ios_show_version`, or an explicit template file. User-defined overrides placed in `~/.local/share/cliscape/templates/` take precedence over embedded defaults.

Furthermore, the embedded library utilizes `rust-embed` to provide dual-mode execution. In release builds, templates are compiled directly into the binary for self-contained template deployment. In debug builds, the engine dynamically reads from the local filesystem, enabling instant hot-reloading of templates during development without requiring recompilation.

5.3 Validation and Edge Case Management

Network CLI data is notoriously inconsistent due to firmware patches, unexpected banners, and hardware variants. The engine implements strict thresholds and boundaries:

- **Coverage Validation:** A configurable warning system triggers if a parsed output matches fewer fields than a minimum acceptable threshold (e.g., 80%), acting as a canary for firmware formatting changes.
- **Timeout Guard:** Rust's regex crate avoids backtracking-style regex explosions, and parse calls can also set an elapsed-time timeout guard for operational control.
- **Snapshot Testing:** A comprehensive internal suite relies on `insta` snapshots, guarding parsed [JSON](#) output against unintended fixture regressions across updates.

- **Execution Modes:** To balance resilience with strictness, the parser supports a Partial-Match mode (gracefully returning successfully parsed data alongside warnings for missing fields) and a `--strict` Fail-Fast mode that aborts execution upon the first error.

5.4 Structured Observability

For production deployments, raw text logs are insufficient. The engine utilises Rust's tracing framework and supports JSON log formatting, capturing module-level lifecycle events, resolution paths, and parsing failures for integration into observability pipelines.

6 Isomorphic Simulation and Synthetic Generation

One architectural direction in `cliscape` is round-trip state machine execution. Traditional parsers represent a unidirectional projection from unstructured text to structured data; `cliscape` adds best-effort generation from structured records back into CLI-like text.

6.1 The Inverse Parser (FSM^{-1})

Because the modern YAML/TOML templates preserve rule patterns and field metadata, the parsing engine can be run in a reverse direction for some templates. In this mode, `cliscape` consumes structured JSON records and uses template rules to synthesize CLI-like text. This is useful, but it is not yet a general inverse-regex solver and should not be described as pixel-accurate for every template.

This capability is a foundation for deterministic network-device simulation. By providing a hypothetical device state, operators can generate representative CLI output for downstream automation tests. The richer simulator, session protocol integration, and vendor-authentic formatting remain planned v3.0 work.

6.2 Mathematical Verification

Round-trip generation also provides a mechanism for template verification. A template is round-trip stable for a fixture if raw CLI text parses into JSON, generates synthetic text, and re-parses into the same JSON. Current verification reports mismatches but does not fail the whole suite for every non-injective template, so stricter oracle semantics remain future hardening work.

7 Interactive TUI Debugger

The built-in Terminal User Interface (TUI) provides an integrated "Live Lab", offering visual, interactive observability into the parser execution, eliminating trial-and-error template development.

Beyond simply debugging a single file, the TUI includes a **Template Browser**. Users can press `t` to browse embedded and XDG user templates, select a template (e.g., `cisco_ios_show_version.yaml`), and load it into the Live Lab alongside their current raw CLI text. The parsing engine re-runs when watched template/input files change, and after saving edits in the template editor.

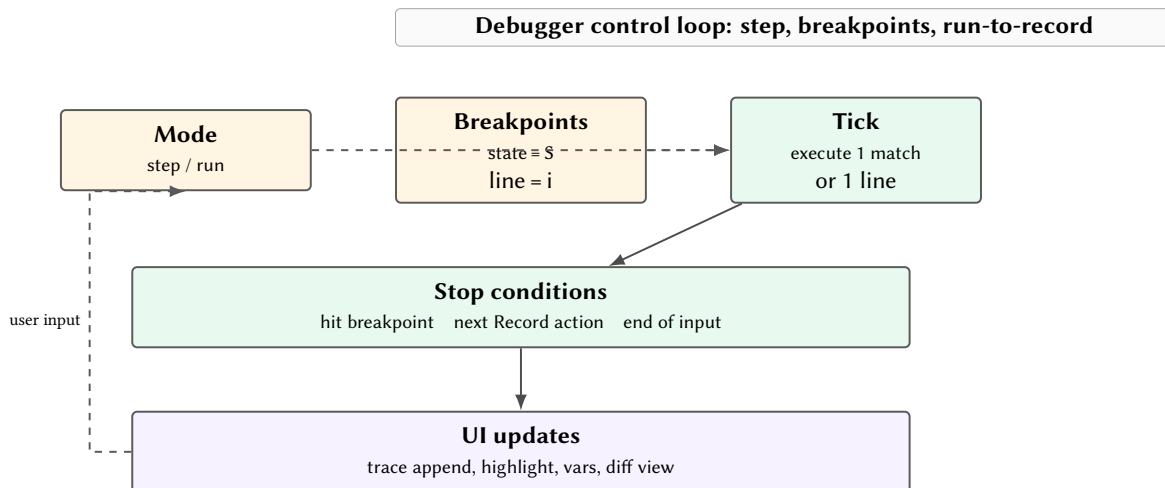


Figure 6. Debugger as a control loop over the parsing interpreter.

The interface offers four panes: an **Input Pane** for raw text with line highlighting, a **State Pane** detailing the FSM transition history, a **Variables Pane** showing live value states, and a **Diff View** explicitly highlighting matching capture groups.

8 Future Trajectories: The Network Compiler

By combining a TextFSM-compatible runtime, typed modern templates, semantic diffing, policy constraints, and best-effort generation, `cliscape` provides a foundation for several advanced applications within the broader automation ecosystem.

8.1 Semantic Drift Analysis

Traditional diff utilities are line-oriented and text-blind, heavily penalizing syntactic noise such as timestamp updates or pagination artifacts. Because `cliscape` projects raw text into structured JSON records, it enables Operational Diffs. By isolating the delta between two parsed states ($S_1 \rightarrow S_2$), the system identifies semantic transitions while ignoring fields explicitly marked as diff noise.

8.2 Vendor-Neutral State Schemas

Network data is traditionally trapped within vendor-specific syntax constraints. `cliscape`'s modern YAML definitions include a `common_schema` field hook that can map vendor-specific names onto a universal semantic schema. The embedded templates do not yet provide full ledger coverage; standard schemas and compliance validation are the Phase 17 Universal Ledger work.

8.3 Grammar Induction and Structural Inference

Current template creation still relies on human refinement. The existing `infer` command can identify static lines shared across samples and emit broad candidate capture groups as a scaffold. Future iterations should deepen that grammar-induction layer over larger corpuses of raw CLI outputs.

8.4 The FSM-Oracle: Self-Verification

The generator facilitates a closed-loop FSM-Oracle direction. When a network device receives a firmware update that alters its CLI output formatting, round-trip checks can detect semantic drift between original and generated records. Today this is a warning-oriented verification loop; a strict CI gate over selected templates is still needed before calling it a proof system.

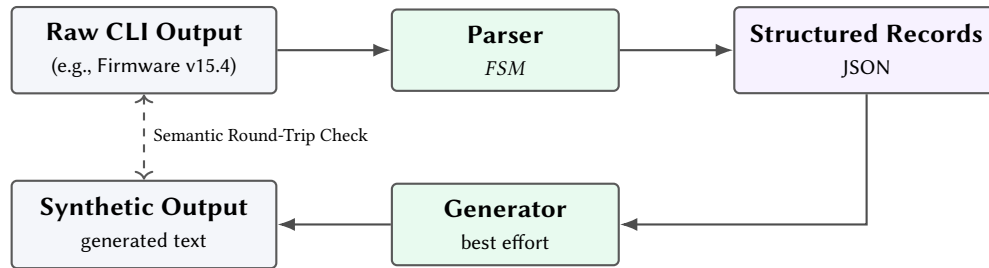


Figure 7. The FSM-Oracle direction. A template is stronger when generated synthetic output re-parses to the same semantic records as the original raw CLI output.

9 Performance Evaluation

The transition from interpreted Python execution to Rust should yield significant performance gains, and the project includes Criterion benchmarks for throughput tracking. Earlier benchmark runs reported approximately **4.1 million lines per second**; current release notes should regenerate this number from the checked-in benchmark suite before making public performance claims.

The compiled Rust architecture and DFA-based regex matching avoid Python GIL bottlenecks and backtracking-style regex explosions. A dedicated performance-regression CI gate and refreshed benchmark baselines remain open follow-up work.

10 Design Summary

Table 1. Key design decisions.

Decision	Rationale
Rust regex crate	DFA-based $O(n)$ matching; no backtracking vulnerabilities
TextFSM compatibility	Migration path for legacy TextFSM templates
Stateful modifiers	Filldown/List support native relational data extraction
Dual template formats	Incremental adoption; YAML for new templates
Native type coercion	YAML template hints yield directly queryable JSON types
Best-effort generation	Structured records can generate representative synthetic CLI text
Embedded library	XDG-compliant, zero-configuration parsing out-of-the-box
TUI debugger	Interactive FSM visualisation reduces trial-and-error template development

References

- [1] Google, *TextFSM: Template-based state machine for parsing*, 2024. [Online]. Available: <https://github.com/google/textfsm>
- [2] Network to Code, *NTC Templates: Textfsm templates for network devices*, 2024. [Online]. Available: <https://github.com/networktocode/ntc-templates>

Revision History

Version	Date	Changes
0.1.0	March 2026	Initial architecture report
0.1.1	March 2026	Added ecosystem context, case studies, production hardening, and performance data
0.1.2	April 2026	Synced claims with current code, tests, and roadmap gaps